

Mochi.dev

**MatchScore
Software Design Document
Version 01.00**

MatchScore	Version: 01.00
Software Requirements Specification	Date: 09/17/2026
<document identifier>	

Revision History

Date	Version	Description	Author
09/17/2026	01.00	Initial Draft	John Michael Villamor

MatchScore	Version: 01.00
Software Requirements Specification	Date: 09/17/2026
<document identifier>	

Table of Contents

1.	INTRODUCTION	4
1.1	Purpose	4
1.2	Scope	4
1.3	Overview	4
1.4	Reference Material	4
1.5	Definitions and Acronyms	4
2.	SYSTEM OVERVIEW	5
3.	SYSTEM ARCHITECTURE	5
3.1	Architectural Design	5
3.2	Decomposition Description	6
3.3	Design Rationale	8
4.	DATA DESIGN	8
4.1	Data Description	8
4.2	Data Dictionary	9
5.	COMPONENT DESIGN	10
6.	HUMAN INTERFACE DESIGN	11
6.1	Overview of User Interface	11
6.2	Screen Images	12
6.3	Screen Objects and Actions	13
7.	REQUIREMENTS MATRIX	13
8.	APPENDICES	14

MatchScore	Version: 01.00
Software Requirements Specification	Date: 09/17/2026
<document identifier>	

Software Design Document

1. INTRODUCTION

1.1 Purpose

This Software Design Document discusses the architecture and system design of **MatchScore**. This document provides clear descriptions of the system's components, data structures, and interfaces in detail. The purpose of this document is to give developers and evaluators an understanding of how the application is structured before and during implementation.

1.2 Scope

The scope of this application covers job seekers who want to build a resume and/or portfolio and check how well it matches a specific job posting. As the application develops, the developers may consider extending support to multi-page portfolio sites, additional file formats, and broader AI-assisted editing features.

1.3 Overview

This portion discusses the overall design of the application in terms of its architecture, data structures, and interface. The application allows a user to build a resume using templates, compare it against a job posting using a deterministic text-similarity algorithm, and receive AI-generated explanations and recommendations to improve their match score.

1.4 Reference Material

- scikit-learn documentation, TF-IDF and Cosine Similarity — accessed 2026, <https://scikit-learn.org/>
- Google Gemini API documentation — accessed 2026, <https://ai.google.dev/>

1.5 Definitions and Acronyms

Term	Description
TF-IDF	Term Frequency–Inverse Document Frequency, a text-weighting technique used to score word importance
Cosine Similarity	A measure of similarity between two text vectors, used to compute the match score
WYSIWYG	"What You See Is What You Get" — an editor that shows final formatting while editing
ATS	Applicant Tracking System, software used by employers to filter resumes
LLM	Large Language Model

MatchScore	Version: 01.00
Software Requirements Specification	Date: 09/17/2026
<document identifier>	

2. SYSTEM OVERVIEW

Job seekers currently rely on separate tools to build a resume and to check whether it matches a job posting — a resume builder (Canva, Google Docs) and a separate "resume checker" tool. This forces users to manually copy feedback from one tool into another, which discourages the improvement step and often results in resumes never actually being checked against the posting they're applying for.

MatchScore addresses this by combining both functions into a single platform. Users build a resume (and optionally a portfolio) from a template directly in the browser, paste a job posting, and receive a deterministically computed match score along with an AI-generated explanation of the result. Missing or weak areas are surfaced as specific, section-targeted recommendations, which the user can apply directly to the document without leaving the platform. The score itself is always computed by an algorithm (TF-IDF and cosine similarity), never generated by AI — this keeps the result reproducible and explainable, with AI's role limited to interpreting and acting on that score.

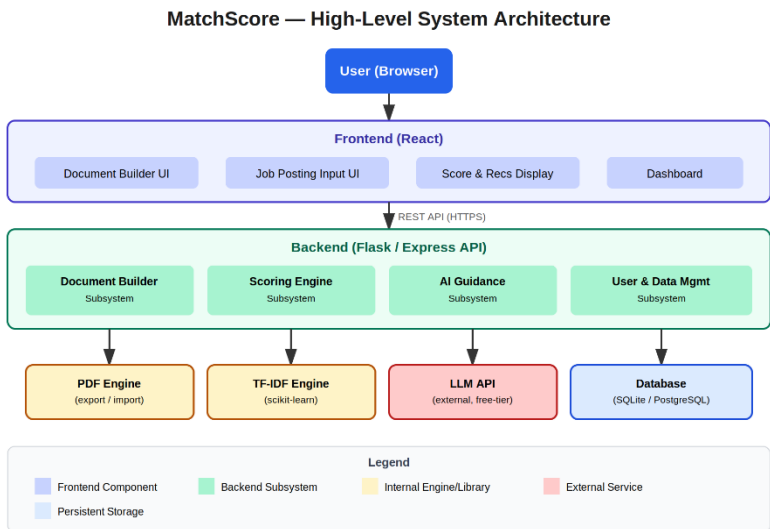
3. SYSTEM ARCHITECTURE

3.1 Architectural Design

The system is decomposed into four major subsystems:

1. **Document Builder Subsystem** — handles template selection, the WYSIWYG editor, PDF import, and PDF export.
2. **Scoring Engine Subsystem** — handles text preprocessing, TF-IDF vectorization, and cosine similarity computation between the resume and job posting.
3. **AI Guidance Subsystem** — handles score explanation generation, section-targeted recommendation generation, and constrained auto-edit rewriting.
4. **User & Data Management Subsystem** — handles authentication, and persistent storage of documents, job postings, match results, and score history.

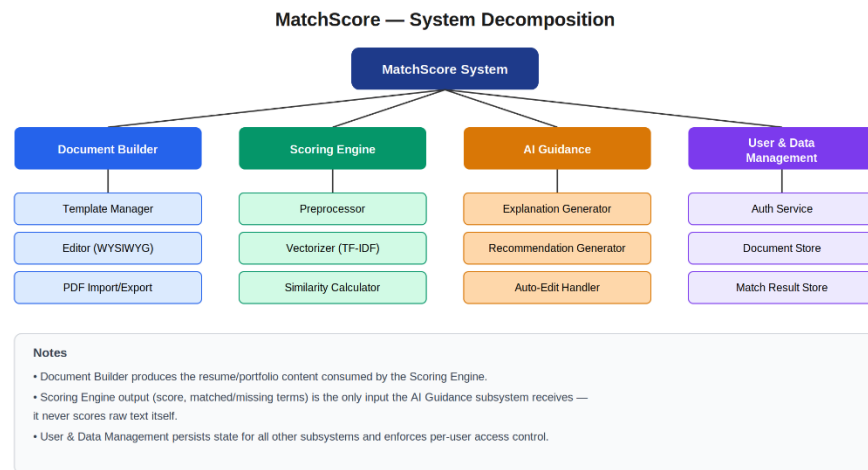
The Document Builder collects the resume/portfolio content and passes it, along with a pasted job posting, to the Scoring Engine. The Scoring Engine computes a deterministic score and a list of matched/missing terms, which are passed to the AI Guidance Subsystem for interpretation. The AI Guidance Subsystem never receives raw, unscored text — it only ever acts on results the Scoring Engine has already computed. All documents, results, and history are persisted through the User & Data Management Subsystem.

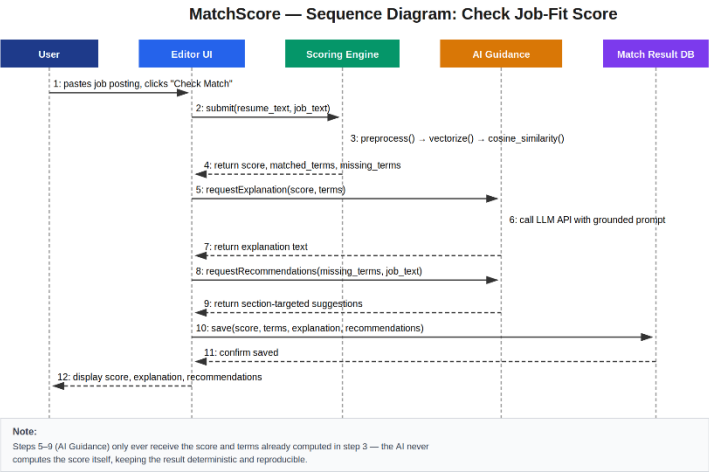
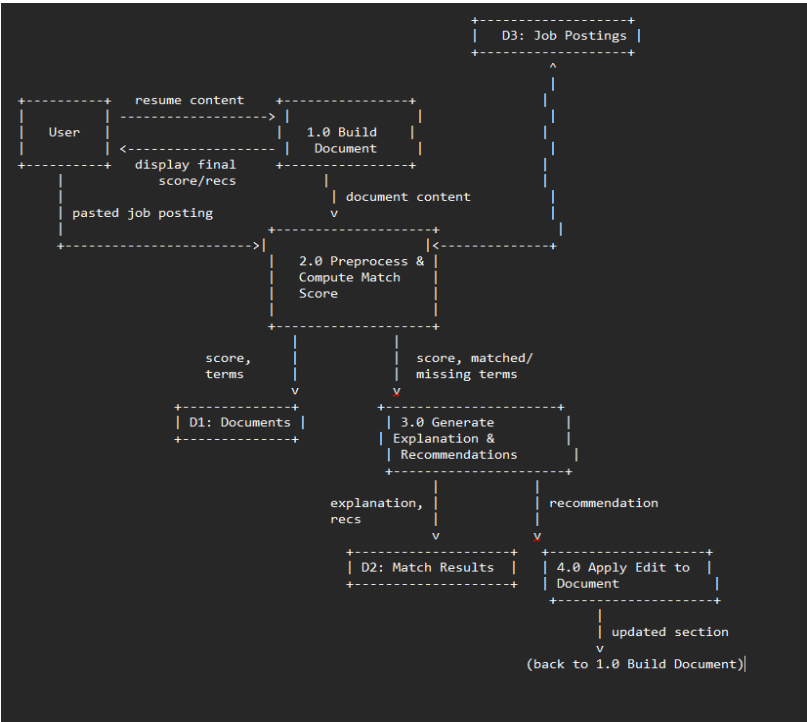


MatchScore	Version: 01.00
Software Requirements Specification	Date: 09/17/2026
<document identifier>	

3.2 Decomposition Description

- **Template Manager** — manages the library of pre-built templates, manually created templates, and AI-generated templates (layout/styling only).
- **Editor** — the in-browser WYSIWYG component, organized into distinct, independently editable sections (Experience, Skills, Education, etc.).
- **PDF Import/Export** — extracts text from an uploaded PDF into the Editor, and renders the finished document to a downloadable PDF.
- **Preprocessor** — tokenizes, removes stopwords, and stems both the resume and job posting text.
- **Vectorizer** — converts preprocessed text into TF-IDF weighted vectors.
- **Similarity Calculator** — computes cosine similarity between vectors and extracts matched/missing terms.
- **Explanation Generator** — takes the computed score and terms, produces a plain-language explanation via the AI API.
- **Recommendation Generator** — produces section-targeted edit suggestions via the AI API.
- **Auto-Edit Handler** — applies an accepted recommendation to the relevant section, constrained to rephrasing existing content only.
- **Auth Service** — handles user registration, login, and session management.
- **Document Store** — persists resumes, portfolios, and version history.
- **Match Result Store** — persists computed scores, matched/missing terms, AI explanations, and recommendations.ay choose to give a functional description or an object-oriented description. For a functional description, put top-level data flow diagram (DFD) and structural decomposition diagrams. For an OO description, put subsystem model, object diagrams, generalization hierarchy diagram(s) (if any), aggregation hierarchy diagram(s) (if any), interface specifications, and sequence diagrams here.





MatchScore	Version: 01.00
Software Requirements Specification	Date: 09/17/2026
<document identifier>	

3.3 Design Rationale

The central design decision in MatchScore is separating the **scoring** step from the **interpretation** step. An earlier design considered letting the AI generate the match score directly (i.e., asking the LLM "how well does this resume match this job?"). This was rejected because LLM outputs are not fully deterministic — the same resume/job pair could produce different scores on different runs, and there would be no way to explain *why* a particular number was produced. Computing the score algorithmically via TF-IDF and cosine similarity guarantees the same input always produces the same output, and gives the AI concrete, ground-truth data (the score, matched terms, missing terms) to explain rather than asking it to invent a judgment.

A second design decision was constraining the AI Auto-Edit feature to rephrasing existing user content only, rather than allowing it to freely rewrite sections. This was chosen to prevent the AI from fabricating skills or experience the user does not actually have — a critical risk for a resume tool, where an untrue claim could seriously harm the user if discovered by an employer.

4. DATA DESIGN

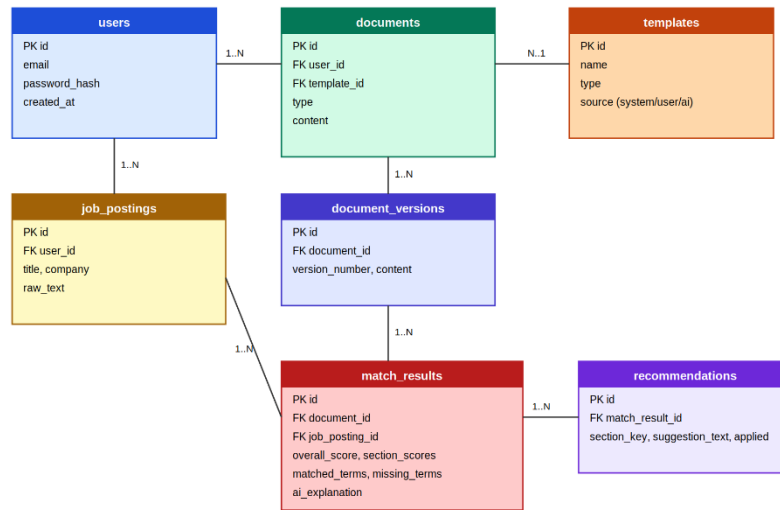
4.1 Data Description

Based on the requirements of the application, the following entities were defined: User, Document, Document Version, Template, Job Posting, Match Result, and Recommendation.

- The entity "**User**" describes an account holder based on the following properties: user ID, email, and password hash. The table `users` was created in the database to describe this entity.
- The entity "**Document**" describes a resume or portfolio based on the following properties: document ID, owning user, type (resume/portfolio), template used, and content. The table `documents` stores this entity, with a foreign key relationship to `users`.
- The entity "**Document Version**" describes a saved snapshot of a document at a point in time, used for autosave and history. The table `document_versions` stores this entity, with a foreign key relationship to `documents`.
- The entity "**Template**" describes a reusable layout, which may originate from the system's default library, be manually created by a user, or be AI-generated from a style description. The table `templates` stores this entity, including a field identifying its source (system/user/AI).
- The entity "**Job Posting**" describes a job listing pasted by the user, based on the properties: job posting ID, title, company, and raw text. The table `job_postings` stores this entity.
- The entity "**Match Result**" describes the outcome of comparing a Document against a Job Posting, based on the properties: overall score, section scores, matched terms, missing terms, and AI-generated explanation. The table `match_results` stores this entity, with foreign keys to both `documents` and `job_postings`.
- The entity "**Recommendation**" describes an individual AI-generated suggestion tied to a Match Result, based on the properties: target section, suggestion text, and whether it has been applied. The table `recommendations` stores this entity, with a foreign key to `match_results`.

MatchScore	Version: 01.00
Software Requirements Specification	Date: 09/17/2026
<document identifier>	

MatchScore — Entity Relationship Diagram



4.2 Data Dictionary

Objects	Type	Description
ai_explanation	text	Match Result attribute, plain-language explanation of the computed score
applied	boolean	Recommendation attribute, whether the suggestion has been applied to the document
company	string	Job Posting attribute, name of the hiring company
content	json	Document attribute, the structured section content of the resume/portfolio
created_at	datetime	Timestamp attribute present on most entities, records creation time
document_id	int	Document attribute, primary key
email	string	User attribute, account email address
job_posting_id	int	Job Posting attribute, primary key
matched_terms	json	Match Result attribute, list of terms present in both resume and job posting
missing_terms	json	Match Result attribute, list of terms present in the job posting but absent from the resume
overall_score	float	Match Result attribute, the computed cosine similarity score (0–100%)
password_hash	string	User attribute, securely hashed password
raw_text	text	Job Posting attribute, the full pasted text of the posting

MatchScore	Version: 01.00
Software Requirements Specification	Date: 09/17/2026
<document identifier>	

section_key	string	Recommendation attribute, identifies which document section the suggestion targets
section_scores	json	Match Result attribute, per-section breakdown of the match score
suggestion_text	text	Recommendation attribute, the AI-generated edit suggestion
template_id	int	Document attribute, foreign key to the Template used
type	string	Document attribute, either "resume" or "portfolio"
user_id	int	User attribute, primary key; also used as a foreign key on Document and Job Posting
version_number	int	Document Version attribute, sequential version counter

5. COMPONENT DESIGN

Preprocess Text (Preprocessor)

- FUNCTION preprocess(text):
- text = lowercase(text)
- tokens = tokenize(text)
- tokens = remove_stopwords(tokens)
- tokens = stem(tokens)
- RETURN tokens

Compute Match Score (Vectorizer + Similarity Calculator)

- FUNCTION compute_match_score(resume_text, job_text):
- resume_tokens = preprocess(resume_text)
- job_tokens = preprocess(job_text)
- resume_vector = tfidf_vectorize(resume_tokens)
- job_vector = tfidf_vectorize(job_tokens)
- score = cosine_similarity(resume_vector, job_vector)
- matched_terms = intersection(top_terms(resume_vector), top_terms(job_vector))
- missing_terms = top_terms(job_vector) - top_terms(resume_vector)
- RETURN score, matched_terms, missing_terms

Generate Explanation (Explanation Generator)

- FUNCTION generate_explanation(score, matched_terms, missing_terms):
- prompt = build_prompt(score, matched_terms, missing_terms)
- explanation = call_ai_api(prompt)
- RETURN explanation
- **Generate Recommendations (Recommendation Generator)**
- FUNCTION generate_recommendations(document, missing_terms, job_text):
- FOR EACH section IN document.sections:
- IF section is relevant to missing_terms:
- suggestion = call_ai_api(build_suggestion_prompt(section, missing_terms, job_text))
- store_recommendation(section.key, suggestion)
- RETURN recommendations

MatchScore	Version: 01.00
Software Requirements Specification	Date: 09/17/2026
<document identifier>	

Apply Auto-Edit (Auto-Edit Handler)

- FUNCTION apply_auto_edit(document, recommendation):
- IF recommendation.requires_new_claim:
- PROMPT user_confirmation
- IF NOT confirmed: RETURN unchanged
- section = document.get_section(recommendation.section_key)
- rewritten = call_ai_api(build_rewrite_prompt(section, recommendation))
- document.update_section(recommendation.section_key, rewritten)
- save_version(document)
- RETURN document

6. HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

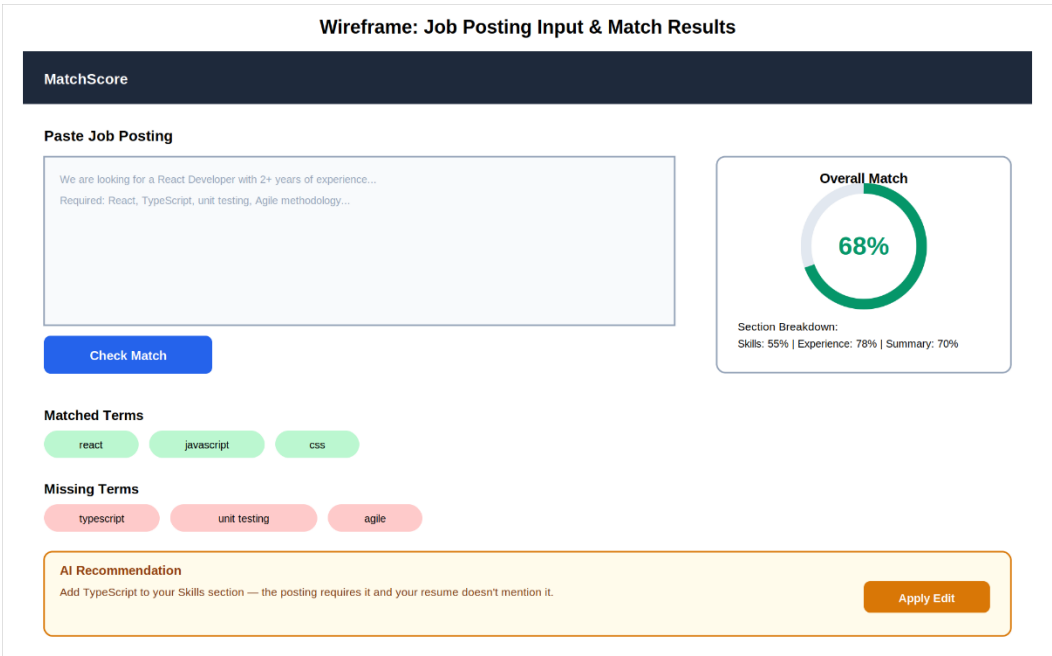
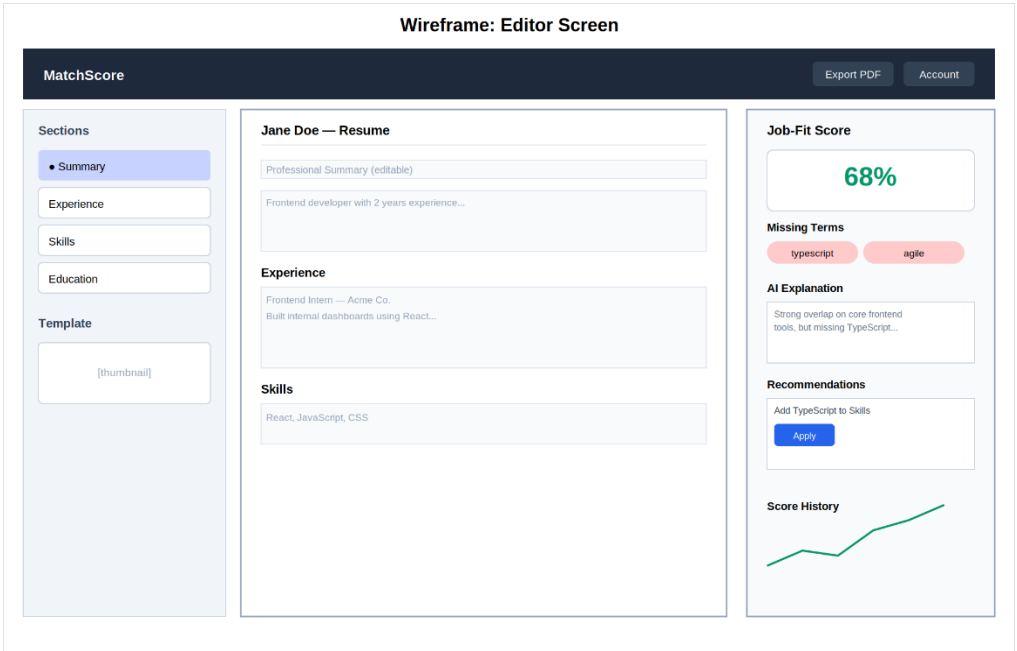
MatchScore is designed to provide a seamless build-and-check experience for job seekers. The software allows the user to select or create a template, edit their resume/portfolio content directly in the browser, and immediately see how well it fits a specific job posting. The features and functionalities of the platform are as follows:

1. Document Builder
 - Choose a template (pre-built, manual, or AI-generated)
 - Edit resume/portfolio content in a WYSIWYG editor
 - Import an existing PDF resume
 - Export the finished document as a PDF
2. Job-Fit Scoring
 - Paste a job posting
 - View the computed match score and matched/missing terms
3. AI Guidance
 - View an AI-generated explanation of the score
 - View section-targeted recommendations
 - Apply a recommendation directly to the document (auto-edit)
 - View score history over time

MatchScore	Version: 01.00
Software Requirements Specification	Date: 09/17/2026
<document identifier>	

6.2 Screen Images

Display screenshots showing the interface from the user’s perspective. These can be hand- drawn or you can use an automated drawing tool. Just make them as accurate as possible. (Graph paper works well.)



MatchScore	Version: 01.00
Software Requirements Specification	Date: 09/17/2026
<document identifier>	

6.3 Screen Objects and Actions

Screen Object	Action
Template Card	Click to select a template and start a new document
Editor Section	Click to edit content inline; autosaves on change
"Import PDF" Button	Opens file picker; uploads and parses a PDF into the editor
"Export PDF" Button	Renders the current document to a downloadable PDF
Job Posting Text Box	Accepts pasted job posting text; submits on "Check Match"
Match Score Display	Shows overall score and section breakdown after submission
Recommendation Card	Displays a suggestion with "Apply" and "Dismiss" actions
"Apply" Button	Triggers the Auto-Edit Handler to rewrite the targeted section
Score History Chart	Displays past match scores for the current document over time

7. REQUIREMENTS MATRIX

Requirement Code	Requirement Description	System Component
3.1.1	The system shall allow users to create an account and log in/out	Auth Service
3.1.2	The system shall provide template library, manual creation, and AI-generated templates	Template Manager
3.1.3	The system shall provide a WYSIWYG editor with autosave	Editor
3.1.4	The system shall allow users to export their resume as a PDF	PDF Import/Export
3.1.5	The system shall allow users to import an existing PDF resume	PDF Import/Export
3.1.6	The system shall allow users to paste job posting text	Job Posting Input (Editor UI)
3.1.7	The system shall compute a deterministic match score	Preprocessor, Vectorizer, Similarity Calculator
3.1.8	The system shall generate a plain-language score explanation	Explanation Generator
3.1.9	The system shall generate section-targeted recommendations	Recommendation Generator
3.1.10	The system shall allow users to apply AI suggestions directly	Auto-Edit Handler

MatchScore	Version: 01.00
Software Requirements Specification	Date: 09/17/2026
<document identifier>	

3.1.11	The system shall track score history over time	Match Result Store
3.5.1	The system shall store passwords securely hashed	Auth Service
3.5.2	The system shall restrict document access to the owning user	Auth Service, Document Store
3.6.1	The system shall operate within free-tier AI API rate limits	Explanation Generator, Recommendation Generator, Auto-Edit Handler

8. APPENDICES

This section is optional.

Appendices may be included, either directly or by reference, to provide supporting details that could aid in the understanding of the Software Design Document.